つくばチャレンジ2024におけるArcanainチームの取り組み
Arcanain Team's Initiatives for Tsukuba Challenge 2024

Abstract– The authors are a team composed of working professionals and graduate students. In this paper, we report on our efforts in the Tsukuba Challenge 2024. In particular, we describe the hardware and software configuration of our mobile robot, the results of our experiments, and the knowledge gained from our experience in Tsukuba Challenge 2024.

Key Words: Jetson Orin Nano, Pure Pursuit, GNSS, SLAM, 2D-Lidar, Odrive

1 はじめに

Arcanainは社会人・大学生・大学院生により構成されたチームである．本稿ではつくばチャレンジ2024における我々の取り組み内容について報告する．特に，我々の自律走行ロボットにおけるハードウェアおよびソフトウェア構成・走行結果・今後の展望について述べる．

2 ハードウェア

本章では，著者らが開発した自律走行ロボットのハードウェア構成について述べる．

2.1 実験機の概要

Fig.1に著者らが開発した移動ロボットの外観を，Table.2に移動ロボットに使用している部品類を示す．部品類は，Amazon，Aliexpress，秋月電子など，入手性が容易な媒体から購入した．搭載している主なセンサは2D-Lidar，GPS，IMU, ホイールエンコーダーである．IMUはアナログ・デバイセズ株式会社様から貸与いただいた．車体はアルミフレームによって拡張が容易な構造にしている．モータードライバーには位置制御や速度制御など電子制御が可能なOdrive[1)]を採用した．PCにはJetson Orin Nano(8GB)を採用し，モバイルバッテリー(DC19V/3.6A)により電源供給した．また，Jetson Orin NanoのOSは，Ubuntu 22.04でROS2 humbleが導入されている．

Table 1: ロボットに搭載されている部品

Components	Model	Number
In-Wheel Motor	Scooter Hub Motor	2
Motor Driver	Odrive v3.6	1
Portable Power Suplly	Anker	1
Lithium ion batteries	BL1860B 18V 6.0Ah	1
2D-Lidar	RPLidar S1	1
IMU	IMU – ADIS16475-2	1
GPS Receiver	ZED-F9P	1
GPS Antenna	ANN-MB-00	1
PC	Jetson Orin Nano(8GB)	1

2.2 ハードウェア構成

移動ロボットの内部ハードウェア構成をFig.2に示す．基本的な構成としては，Jetson Orin Nanoに2D-Lidar，GPS, IMU, モータードライバーなど各種デバイスを接続した構成となっている．自己位置推定の入力情報として, 2D-Lidar, GNSS, IMUを用いている. 前方の障害物は2D-Lidarにて検知した. モータードライバーはインホイールモーターの駆動系制御にそれぞれ使用した．Odrive[1)]は2つのBLDCモーターを電子制御するオープンソースのDCモーターコントローラである．マキタバッテリー(BL1860B 18V 6.0Ah)を専用アダプターによりDC出力(24V, 5.0A)に変換して電源供給している．

Fig. 1: 自律移動ロボット (Arumiちゃん)

3 ソフトウェア

本章では，著者らが開発したソフトウェア構成について述べる．

3.1 ソフトウェア構成

本システムのソフトウェア構成をFig.4，Fig.5に示す．本システムは，事前の経路作成と(Fig.4)と自律走行(Fig.5)によって構成される．以降の節で，自己位置推定，経路追従制御，障害物回避, 駆動系制御の詳細についてそれぞれ述べる．本システムは，外界環境に応じて，2D-LidarとGNSSを切り替えて自己位置推定を行った. その使い分けについても後述する．

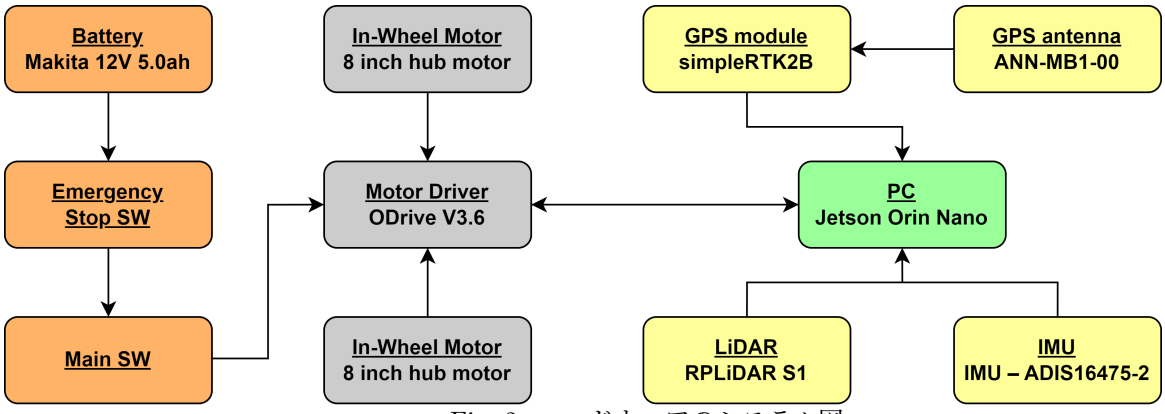

Fig. 2: ハードウェアのシステム図

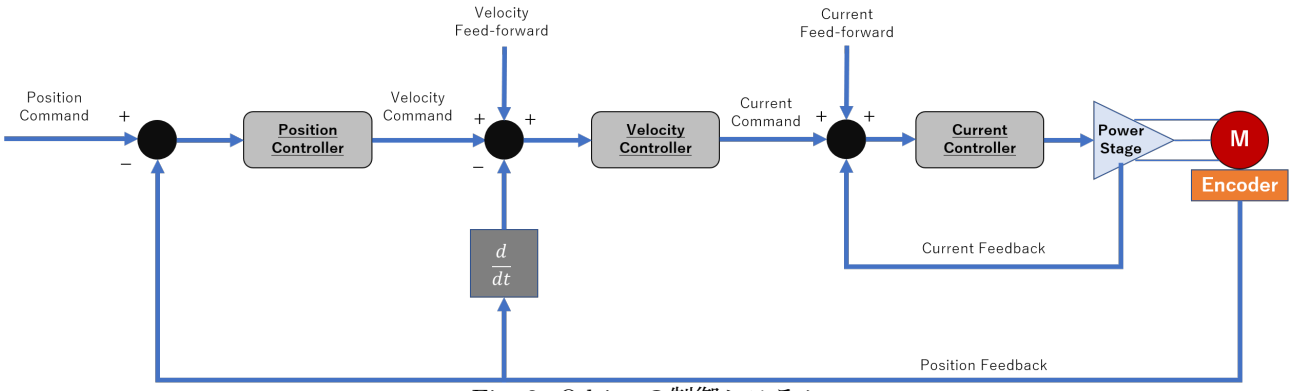

Fig. 3: Odrive の制御システム

3.2 自己位置推定

3.2.1 GNSS・IMU による自己位置推定

本節では，GNSS の座標情報を用いて姿勢角を推定する手法[8]と，IMU の角速度計測値による推定を組み合わせることで，ドリフト誤差や GNSS 信号状態の変動に強い自己位置推定を行う方法を述べる．

まず，GNSS を用いた姿勢角の推定について述べる．RTK 測位[1]により fix 解を得られる場合は，誤差数センチの高精度な緯度・経度情報が取得できるため，短い時間 i をおいて計測した 2 点の GNSS 座標から進行方向（ヨー角）を推定できる．図 7 のように，時刻 $t-i$ での GNSS 座標を (x_{t-i}, y_{t-i})，時刻 t での GNSS 座標を (x_t, y_t) とすると，その差分からロボットの進行方向 θ を以下のように求める．

$$\begin{aligned} \Delta x &= x_t - x_{t-i} \\ \Delta y &= y_t - y_{t-i} \end{aligned} \tag{1}$$

ここで，Δx は東西方向への移動量,Δy は南北方向への移動量である，これにより，

$$\theta_t = \tan^{-1}\left(\frac{\Delta y}{\Delta x}\right) \tag{2}$$

[1]ソフトバンク株式会社/ALES 株式会社が提供している RTK 補正データ配信サービス（SB サービス名：高精度測位サービス ichimill（イチミル）/ALES サービス名：センチメートル級測位サービス）を使用した

上記のようにしてロボットの進行方向が得られる[2]．

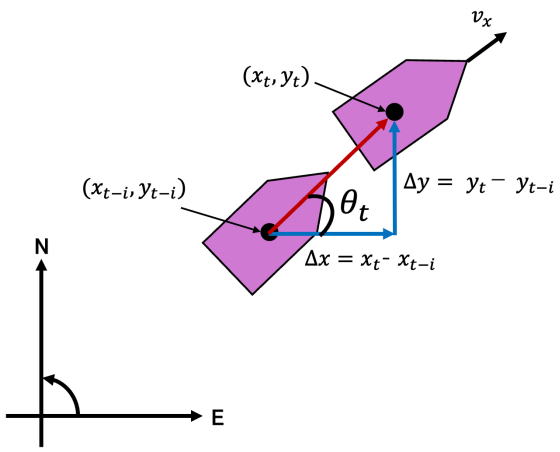

Fig. 6: GNSS 座標によるロボットの姿勢推定

GNSS を用いた姿勢角の推定は，走行中に以下の条件を要求する．

1. RTK 測位が fix 解を維持していること *1
2. ロボットがある程度の速度で走行していること（0.2m/s より速い）*2

[2]尚，本手法はロボットが非ホロノミック（Non-holonomic）型であることを前提としている．速度ベクトルの角度を姿勢角として用いて推定を行なっているため，進行方向と移動体の姿勢角が一致する必要がある．

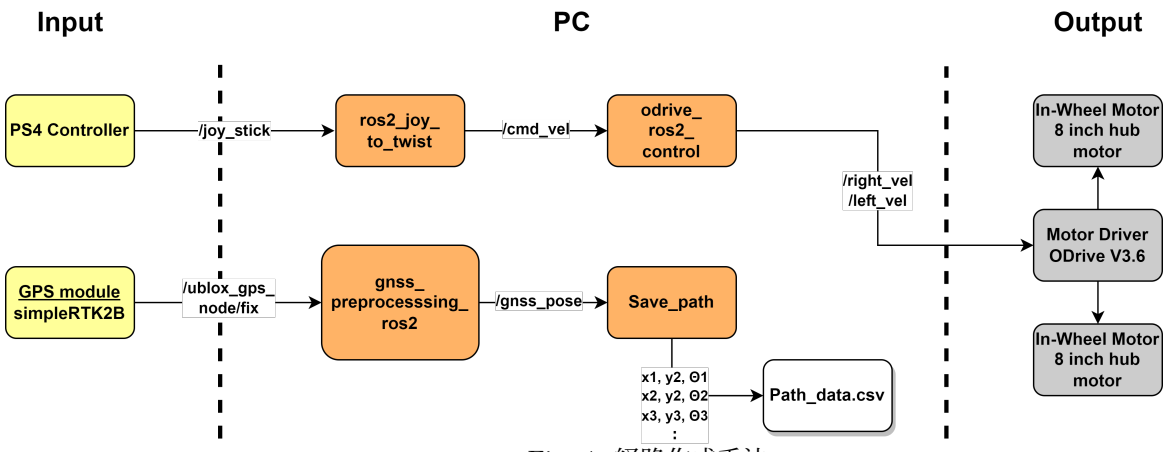

Fig. 4: 経路作成手法

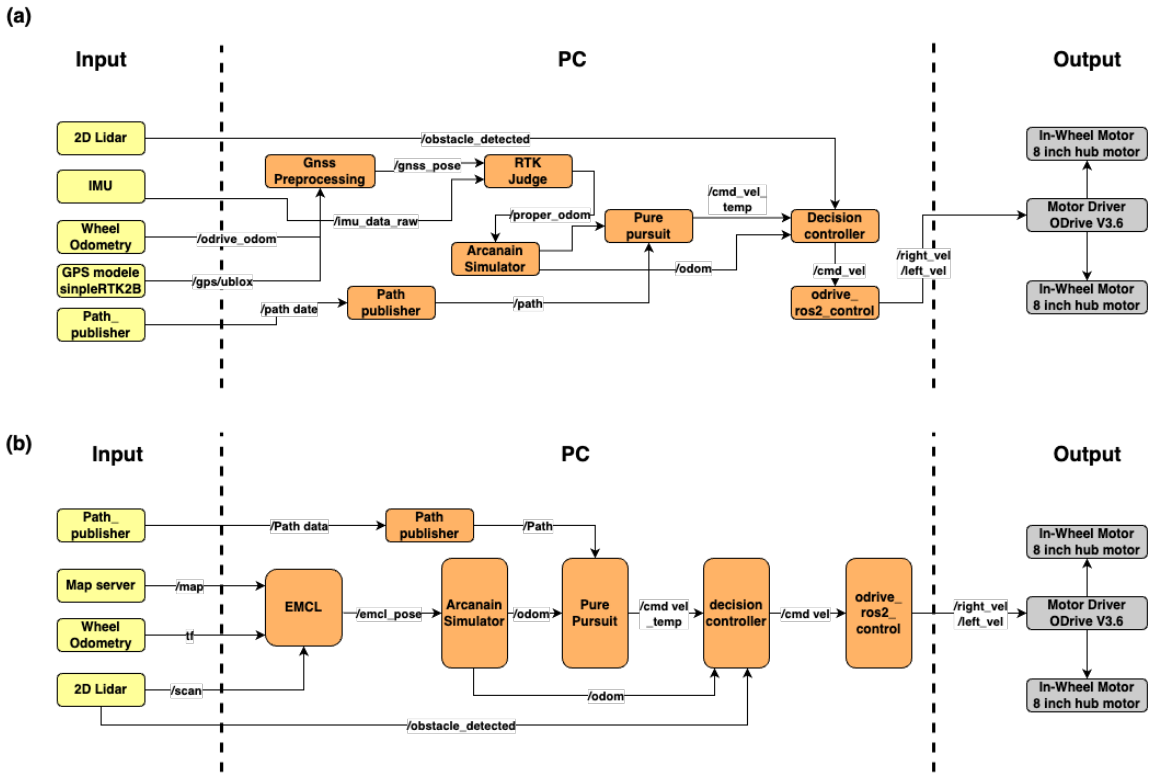

Fig. 5: 自律移動のシステム概念図, (a) 自己位置推定を GNSS を用いて行った場合, (b) 自己位置推定を EMCL を用いて行った場合

実際の走行環境では，建物や障害物の影響などで常に fix 解が得られるとは限らない．また停止や低速走行時には値が発散するため使用できない．

IMU の角速度を積分する方式は，長時間の運用でドリフト誤差が蓄積する，外界の磁場等による一時的な誤差も無視できないという課題があるが，短時間であれば精度よく姿勢を推定できる．この性質を利用し，図 7 に示すフローチャートのように，条件 1，2 が満たされる場合のみ GNSS から得られる姿勢角を採用し，条件を満たさない場合は IMU 由来の推定値を用いる．さらに，GNSS の座標差分が 3m を超える（屋外環境でのマルチパスや受信不良など）場合や 0.001m よりも小さい場合*3, また GNSS で推定した姿勢角が急激に変動 ($\pi/5$ を超える) して連続性が失われる場合*4 などには IMU 推定に切り替えることで，大きな外乱を排除しながら安定した姿勢角推定を実現した．また，この GNSS および IMU を組み合わせてロボットの姿勢角を推定するアルゴリズムは，`RTK_Judge`（Fig.5）によって実装した．

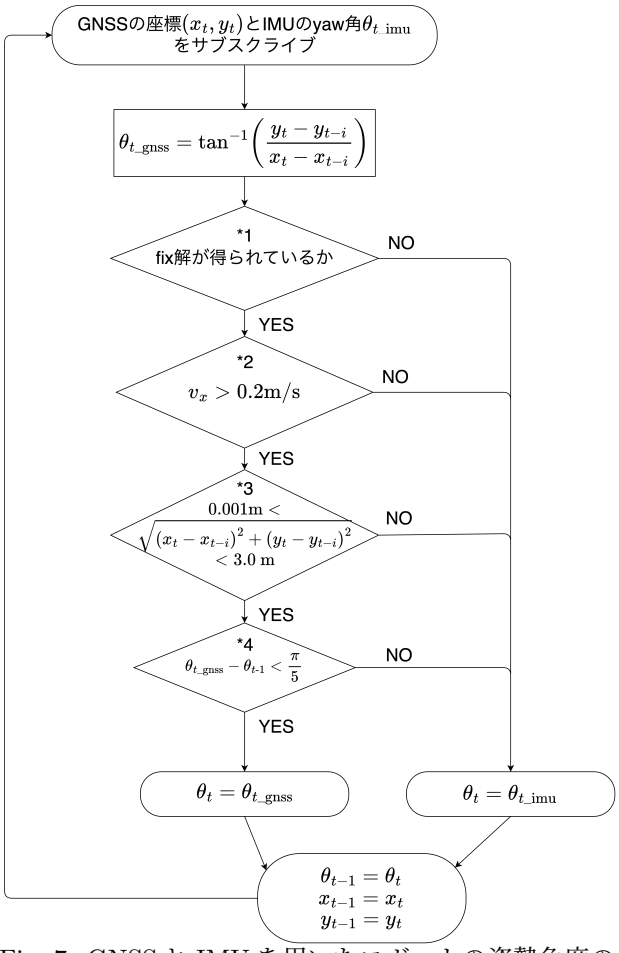

Fig. 7: GNSS と IMU を用いたロボットの姿勢角度の推定方法を記述するフローチャート，x_t, y_t: t 秒時点の GNSS の座標, θ_{t_imu} : t 秒時点の IMU の姿勢角度, θ_{t_gnss} : t 秒時点の GNSS で推定した姿勢角度, v_x : ロボットの並進速度

3.2.2 EMCL による自己位置推定

本節では，Lidar を用いた EMCL（Extended Monte Carlo Localization）[4] による自己位置推定について述べる．

EMCL とは，AMCL（Adaptive Monte Carlo Localization）を基盤として改良が加えられたパーティクルフィルタベースの自己位置推定手法である．AMCL では，通常ロボットの動作（オドメトリ）とセンサ観測が同期待ちになりやすく，ロボットが停止中にパーティクル分布が更新されにくいなどの課題があった．一方，EMCL では，オドメトリの更新を一定周期で行い，センサ情報は非同期に取り込んで重みを更新・リサンプリングすることで，ロボットが停止中でも微調整され続ける仕組みを導入している．さらに，重みの総和がしきい値を下回った際には膨張リセットを行うなど，自己位置が発散にくい実装が施されている．

EMCL を用いて自己位置を推定するには，あらかじめ周囲の環境地図を用意する必要がある．我々は，2D LiDAR を使用し，SLAM パッケージとして Cartographer を用いて 2 次元の占有格子点地図を作成した．作成した地図を Fig.8 に示す．地図作成手順の詳細を公開してある [3] ので，参照されたい．

Fig. 8: cartographer を用いて作成した屋根裏付近の地図

走行時，上記手順で作成した地図を用いて EMCL によって自己位置推定を行なった．ROS2 humble における EMCL は公開されている [6]．EMCL の詳細なアルゴリズムは上田 隆一 (2019)[7] を，実機への導入に関しては我々の公開した記事 [5] を参照されたい．

3.2.3 区間に応じた自己位置推定手法の切り替え

今年度の走行経路には，屋根や建物の影響で電波が遮蔽され，GNSS を用いた自己位置推定が困難な区間（non-fix area）が含まれていた．

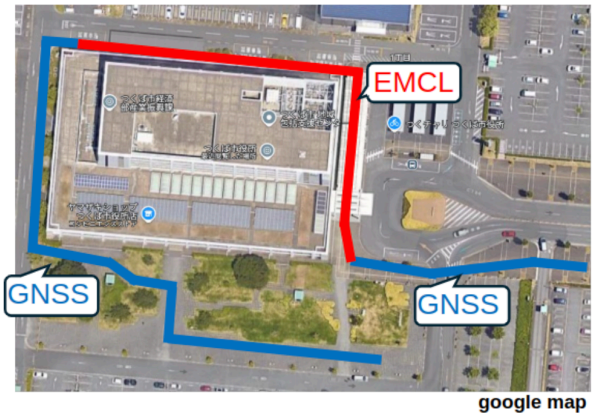

Fig. 9: GNSS もしくは EMCL で自己位置推定を行う区間

3.2.1 節で述べたように，屋根がない開けた領域（fix area）では RTK 測位による GNSS ベースの自己位置推定が有効である．一方 GNSS ベースの手法は fix 解が得られない non-fix area において位置・姿勢推定が不安定になるため，代わりに Lidar を用いた自己位置推定手法が有効である．

我々は，GNSS ベースと EMCL の 2 つの自己位置推定手法を 1 つのシステムで併用することを目指した．しかし実装する際に, `tf` (`map`, `odom`, `base_link` の座標関係) の干渉が起きた．これは GNSS ベースの自己位置推定では，

$$\begin{aligned} \texttt{/tf_fix} : \texttt{map} &\rightarrow \texttt{odom}, \\ \texttt{/tf} : \texttt{odom} &\rightarrow \texttt{base_link} \end{aligned} \tag{3}$$

であるのに対し，EMCL では

$$\texttt{/tf}:\texttt{map} \rightarrow \texttt{odom} \rightarrow \texttt{base_link} \tag{4}$$

であるからである．GNSS ベースの手法は map，odom 間を静的な座標変換として扱う一方，EMCL 手法では全てを動的に変換しながら自己位置推定を行う，そのため，同一システムに共存させることが困難であった．そこで我々は, Fig.9 に示すように，事前に経路全体を GNSS による自己位置推定が可能な領域（fix area）と困難な領域（non-fix area）に分類[8] した．さたに GNSS ベースの手法 (Fig.5, (a)) と EMCL(Fig.5, (b)) を完全に分離した．システムごとに個別の launch ファイルで管理することで，実行単位でシステムを分割でき，システムの切り替え時には，一方のシステムを停止し，もう一方を起動することで干渉を完全に防ぐことができた．

3.3 経路計画

我々は事前に自律走行する経路を作成し，経路追従を行うオフライン経路計画を採用した．以下に経路作成方法について述べる．

経路作成方法のソフトウェア構成を Fig.4 に示す．まず，自律走行する環境にて，PS4 コントローラで移動ロボットを手動で走行させる．つぎに，移動ロボットが 0.1m 進むごとに自己位置推定結果を csv ファイル (Path data.csv) に保存する．実際に保存された経路の様子を，Fig.10 に示す．Fig.10 に示す経路が移動ロボットの目標経路として設定され，次節で述べる手法に基づいて逐次追従することで自律移動を達成した．

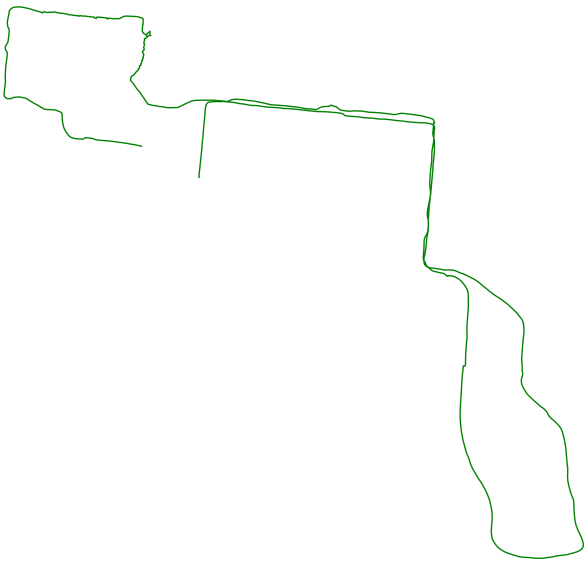

Fig. 10: GNSS で作成したつくばチャレンジの経路

3.4 経路追従制御

本節では，著者らの経路追従制御について述べる．ここで，経路追従制御のソフトウェア構成を Fig.5 に示す．まず，事前生成した csv ファイルから目標経路を生成する．つぎに，3.3 節で述べた自己位置推定を実行し，移動ロボットの位置を算出する．

3.4.1 Pure Pursuit 法

本実験では，Pure pursuit 法に基づいて移動ロボットを目標経路に追従させる．ここで，Pure pursuit 法とは, 並進速度 v_r 及び前方注視距離 l_d に対し, 並進速度 v_r で移動ロボットの現在位置 (x_r, y_r) から l_d 先の経路上の点，前方注視位置 (x_l, y_l) へ向かうための旋回制御手法である．Pure pursuit 法を用いて, 制御入力 v, ω と旋回半径 R をつぎのように与える．

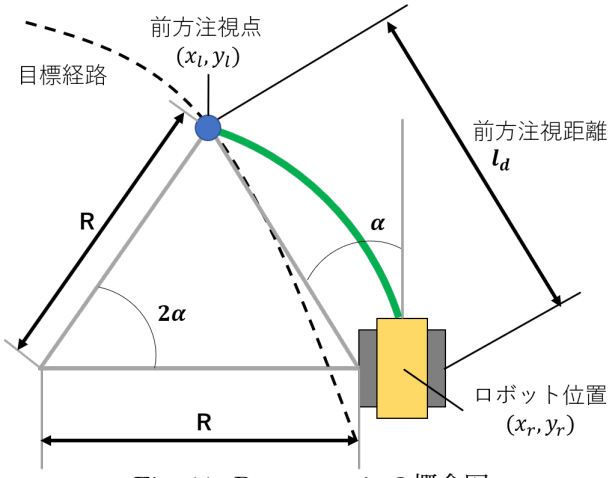

Fig. 11: Pure pursuit の概念図

$$\begin{aligned} v &= v_r \\ \omega &= \frac{2v\sin\alpha}{l_d} \\ R &= \frac{v}{\omega} = \frac{l_d}{2\sin\alpha} \end{aligned} \tag{5}$$

ここで, α は次のような, 目標方向ベクトルと現在の方向ベクトルのなす角である．

$$\alpha = \tan^{-1}\left(\frac{y_l - y_r}{x_l - x_r}\right) - \theta \tag{6}$$

3.5 前方障害物の処理

2D-LiDAR でロボット前方の物体を検出した際には, 停止することで衝突を未然に防いだ. ただし, 壁やポールなどの周囲の静的障害物が原因で停止すると, そのまま動かなくなる可能性があるため, 一定時間停止した後は自動的に走行を再開する仕組みを取り入れている．確認走行区間では, この停止ロジックのみを用いて走行したが, その後の区間ではコーンを避けながら走行する必要があったため, 図 12 に示すような簡易的な障害物回避ロジック（障害物を検出→停止→右に旋回→予定ルートへ復帰）を採用した. 障害物回避の動作は decision_controller（Fig.5）が担っており, 障害物検知トピック（/obstacle_detected）を受け取ると, 回避ロジックに基づいた/cmd_vel を出力することでロボットの回避行動を制御した．

3.6 横断歩道前の停止

今年度選択課題 B の信号認識による横断-待機判断は行わず，横断歩道前の白線停止のみ取り組んだ．本節

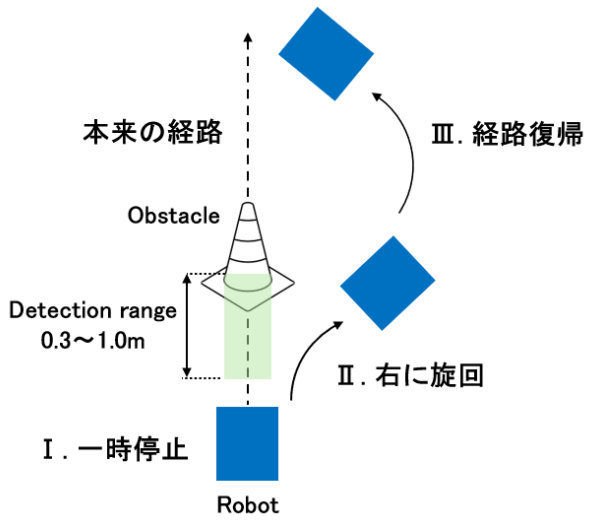

Fig. 12: 障害物回避の手法

では白線停止方法について説明する．

実験走行により，白線停止地点は RTK 測位による fix 解が得られることが確認されたため，事前に停止地点を高精度で特定することが可能であった．白線停止地点それぞれにおいて Fig.4 と同様に数秒間座標を csv ファイルに保存した．数秒間保存された csv ファイルの点は停止ポイントを表す．次に，Fig.10 に示した経路を走査し，経路上で，停止ポイントと最も近い点を特定した．

走行時の停止動作は，`decision_controller`（Fig.5）によって実装した．走行時，`decision_controller` は `odrive_ros2_control` に指令値を/cmd_vel トピックで渡す．

`decision_controller` は，特定した点を保有し，/odom トピックによって現在の移動体の座標も監視する．移動体と停止ポイントの距離があらかじめ設定した閾値 0.5 m に達した場合に速度 0 を指令値として/cmd_vel で出力することによって，所望の位置近傍で停止した．

3.7 駆動系制御

本節では，著者らの駆動系制御方法について述べる．本実験では，駆動系制御のために Odrive を採用した．Odrive の制御方式には，位置制御，速度制御，トルク制御など様々な制御方式が存在し，ユーザーは Odrive のコンフィグレーションファイル内で制御方式を指定することが可能である．ここで，Odrive の制御構成を Fig.3 に示す．位置制御に設定した場合，Fig.3 の閉ループ系全体が実行される．速度制御に設定した場合，位置制御部分が取り除かれ，速度指令値が速度制御部に直接入力される．トルク制御に設定した場合，電流制御部のみ実行される．各閉ループ系は PID 制御により実装されているため，制御パラメータのチューニングが必須となる．本実験では左右輪それぞれに対する目標回転角速度を指令値として送信するため，速度制御を採用した．ここで，左右の車輪の回転角速度 ω_L, ω_R は以下の式で表される．

$$\begin{cases} \omega_L = \dfrac{v - d\omega}{r} \\ \omega_R = \dfrac{v + d\omega}{r} \end{cases} \tag{7}$$

ここで，r は車輪の半径，d はロボットの中心から車輪までの距離である．また，v，ω は 3.5 節で述べる経路追従手法により算出される目標並進速度，目標回転角速度である．この ω_L, ω_R に左右の車輪の回転速度が追従するよう制御する．また，本実験で使用したような安価なモーターの場合，ホールセンサの信号線にノイズが混入することがあるため，ホールセンサと GND の間にセラミックコンデンサーやフィルムコンデンサーを導入することで，信号を滑らかにする必要がある．ホームセンサの信号が滑らかでないと適切なフィードバックが行われず制御不能となる可能性があるので注意が必要である．さらに，本実験で使用したコンフィグレーション自動書き込みスクリプト及び，ROS とのインターフェーススクリプトのコードを公開している[2] ので，適宜参照していただければと思う．

4 結果・考察

つくばチャレンジ 2024 における著者らの本走行は，確認走行区間を超え市役所敷地の入口（650m）にて終了という結果に終わった．

今年度大会における我々の目標は，事前作成した経路を追従することによって全経路を完走することであった．昨年度までの我々の結果を踏まえ，まずは確認走行区間を安定して完走できるハードウェアとソフトウェアを開発することに注力してきた．

走行結果は，著者らのチームの本走行の結果としては最長のものであり，確認走行区間を突破することができた．昨年度ハードウェアとソフトウェアの観点からまとめる．最後に本走行失敗の要因をまとめて本章を結ぶ．

4.1 ハードウェア

ハードウェアに関しては，昨年度からタイヤの半径も大きくしている．これらは 2021,2022 年度本走行におけるスタック（Table.2）からの反省である．昨年度は制御系の不具合で実機がうまく動作せず，アップデートによる効果を検証できなかった．今年度，実験走行，本走行通して点字ブロックや芝生エリアにおいてスタックすることなく安定して走行することができた．これらの結果から，ハードウェアのアップデートは効果的であったと考えられる．

4.2 ソフトウェア

今年度のアップデートにより，一定の性能向上が見られた．以下にまとめる．

4.2.1 GNSS+IMU による姿勢推定の効果

3.2.1 節で述べたように，GNSS を用いることで fix area においては高精度の位置推定が可能である．この手法は，条件を満たしさえすればほとんど誤差の蓄積が発生しないことが利点である．実験走行ではホイールオドメトリを用いた姿勢角推定も実施し，比較を行

なった．ホイールオドメトリの回転数のみで姿勢角推定を行なった場合，経路を進むにつれ無視できないほど大きな誤差が蓄積した．一方，本手法を採用した本走行では，誤差の蓄積も発生せず，非常に安定して走行することができた．

4.2.2 Lidar の性能と EMCL の精度

GNSS が使用できない屋根下の区間（non-fix area, Fig.9）で EMCL による自己位置推定を行なったのは 3.2.2 節で述べたとおりである．EMCL の導入以前はホイールオドメトリのみで自己位置推定を行なっていたが，実験走行では一度も区間を完走することは出来なかった．本走行での走行結果からも，Lidar を用いた自己位置推定が非常に有用であることが確かめられた．

Lidar の性能と EMCL のパフォーマンスの関係について，実験走行をもとに考察する．本走行で使用した Lidar は RPLidar S1（Table 1）で，点群の最大検知距離は約 40m である．また 11/23(土) までの実験走行では最大検知距離が約 20m の RPLidar A1 を用いていた．

我々は初め Lidar A1 を用いて走行することを目指していた．しかし A1 モデルの性質上，太陽光下で点群取得の精度が著しく落ちた．EMCL から GNSS に切り替えるポイントでは，太陽光の影響で自己位置推定が破綻してしまう問題に直面した．そこで太陽光耐性に定評のある S1 モデルを採用した．S1 モデルによって上記の課題は克服された．一方，太陽光の問題は解決したものの，事前の地図作成において課題が残った．

今年度は，新たに S1Lidar を用いて地図を再構築する余裕がなかったため，従来の検知距離 20m で作成した地図を継続利用することにした．結果として，屋根下区間では概ね安定した走行が可能だったものの，EMCL から GNSS へ切り替える区間では地図情報が不足し，マッチング精度が大きく低下してしまった．市役所前のロータリー中央部の形状は，20m の検知範囲では十分に捉えることができなかったが，40m であれば把握可能であった．今後は検知範囲を拡張した地図を新たに作成することで，マッチング精度のさらなる向上が期待できると考えている．

4.2.3 ２つの自己位置推定の切り替え

3.2.3 節で述べたように，我々は GNSS ベースと EMCL の 2 つの自己位置推定システムを完全に分離して管理した．さらに，事前に決めた経路の領域ごとにプロセスを切り替えることで，座標変換による干渉を回避した．

この方法は，2 つのシステムが互いに干渉することを防ぐうえで有効な手段であった．実験走行での試行錯誤によって，適切なタイミングでの切り替え処理を実装できたことが本走行につながったと言える．

一方，この方法では切り替えのたびに `launch` ファイルを再起動する必要がある．プロセス内では，それぞれのシステムをサブシステムとしてジョブを投入・停止（kill）することで切り替えを実現している．そのため，切り替え時に遅延やオーバーヘッドが生じる．また切り替えのタイミングではロボットは完全に処理を停止しており，動作できない，実験走行時において，プロセスの切り替え中に他のロボットが接近することがあった．今回採用した手法は，確かに 2 つのシステムを分離しながら使い分けるためには有用であったが，自律動作システムとしては柔軟性に欠けるものであると考えている．来年度以降，２つの手法の共存・柔軟な切り替えに取り組みたい．

4.3 本走行完走失敗の要因

本走行は約 600m の距離を走行し，市役所敷地の入口付近で終了となった．その要因は，launch 切り替え時に pure pursuit 法で用いる注視点が大きくずれてしまったためである．本来，注視点はロボットと経路の最短距離を求めたうえで，その 1m 先（経路上）に設定される．しかし，launch を切り替える際にバグが発生し，ロボットではなくスタート地点との最短距離を誤って計算してしまったのだ．実はこの問題，実験走行時には修正済みだったものの，本番では古いコードを誤用してしまったことが原因だった．

この一件を通じて，ソフトウェアを GitHub で管理する際には，どのブランチから派生したものかを的確に可視化し，逐次マージして運用する必要性を改めて痛感した．

Table 2: Experimental results

年度	走行記録	失敗要因
2021	100m	芝生にてスタック
2022	250m	点字ブロックにてスタック
2023	10m	意図通りに速度制御が機能しなかった
2024	650m	市役所敷地入口にて経路を外れて失敗

5 展望

つくばチャレンジ 2025 に向け, まずは完走を目指すという目標を念頭に置きながら, ロボットの自律走行技術をさらに高めていきたい. その中で, 特に画像認識および経路計画に注力することで, 以下のような機能の実装と改善を図る.

まず, カメラによる路面認識を行うことで, 路面マーカーや障害物との位置関係をリアルタイムで把握できる仕組みを目指す. また, 障害物に関しては, 単に有無を判定するだけでなく, 障害物の大きさや種類を推定するアルゴリズムを導入することで, より柔軟な回避行動を実現する. これにより, 周囲にいる歩行者や自転車, 車いすなどの識別にも対応し, 安全性の高い走行が可能となることを期待している.

さらに, 信号機認識に挑戦することも重要な課題である. 現時点で YOLO を活用した認識アルゴリズムを作成済みであり, 来年度には ROS2 環境への組み込みを試みる予定だ.

今年度は, 障害物回避アルゴリズムの開発に十分な時間を確保することができず, あらかじめ決まった動作のみでの回避を行っていた. しかしながら, 来年度以降は, 動的な障害物や複雑な交差環境にも対応できる DWA (Dynamic Window Approach) などの経路計画手法を組み込み, 柔軟かつ安全な走行ができるよう改良を進め

ていきたい.

また, 自己位置推定においては,EMCL と GNSS を単に launch ファイルで制御システムごとに切り替えるのではなく, Life Cycle Node を活用することで特定のノードを停止・起動させるシステムを構築し, より汎用的なシステムの構築を検討している. これにより, 電波環境が変化する場所や GNSS が安定しないエリアでも, 最適な自己位置推定アルゴリズムを状況に合わせて切り替え, 安定した自律走行を連続的に継続できるようになると考えている.

今後は, これらの改善点を一つずつクリアし, カメラを活用した総合的な認識と柔軟性の高い経路計画システムを確立することで, つくばチャレンジ 2025 における安定走行・完走の実現を目指していく.

参考文献

1) odriverobotics, "ODriveDocumentation", https://docs.odriverobotics.com/
2) Arcanain/odrive motor control, https://github.com/Arcanain/odrive_motor_control
3) Cartographer で地図を作成する (ROS2 humble), https://qiita.com/tomtomApp/items/8d29a2dd00e60dade9e8
4) 真面目に amcl の代替 ROS パッケージを作った, https://b.ueda.tech/?post=20210505
5) Lidar を用いた自己位置推定 (EMCL)ROS2 humble, https://qiita.com/tomtomApp/items/da3d37f41f334d34a599
6) emcl: mcl with expansion resetting, https://github.com/ryuichiueda/emcl
7) 上田 隆一 (2019), 詳解 確率ロボティクス Python による基礎アルゴリズムの実装
8) 中村勇太, GNSS と 2D 地図の位置推定切替機構を備えた クローラロボットナビゲーションシステム, つくばチャレンジ 2023 参加レポート集, 2023.
9) 築城 大知, 坂口 聡範, 越智 亮太, 山本 薫: Pure pursuit 法を用いた移動ロボットにおける滑らかな障害物回避手法の提案, 第 64 回自動制御連合講演会, 746/750(2021)
10) Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi: You Only Look Once: Unified, Real-Time Object Detection, In Computer Vision and Pattem Recognition(2016)
11) Arcanain/process_supervisor https://github.com/Arcanain/process_supervisor/tree/feature/performance2024
12) IT で社会の役にたちたい | 株式会社 GxP, https://www.gxp.co.jp/
13) Arcanain, https://github.com/Arcanain